

RAPPORT DE PROJET DE FIN DE FORMATION

FITZONE

Conception et Réalisation d'une Plateforme Web Interactive de Gestion Administrative et de Pointage Physique pour Salle de Sport

Présenté en vue de l'obtention du Diplôme de Technicien Spécialisé en Développement Digital

RÉALISÉ PAR

Abdessalame CHRAIBI

Stagiaire — Développement Digital

ENCADRANT

Monsieur DAIF

Formateur — ISAG OFPPT

DÉDICACES

À mes chers parents,

Pour leurs sacrifices sans limites, leurs encouragements constants et leurs prières tout au long de ma formation à l'OFPPT.

À mes frères et sœurs,

Pour leur soutien indéfectible et leur confiance en mes capacités.

À mes formateurs de l'ISAG,

Qui ont guidé mes pas sur le chemin du savoir numérique et du développement web.

À mes collègues stagiaires,

En souvenir des moments partagés de travail et d'entraide tout au long de cette formation.

REMERCIEMENTS

Avant tout développement dans ce rapport, il nous semble opportun de commencer par remercier ceux qui nous ont beaucoup appris au cours de cette formation, ceux qui nous ont accompagnés dans nos travaux et ont toujours su répondre à nos interrogations techniques.

Nous tenons en premier lieu à exprimer notre profonde gratitude et nos sincères remerciements à notre encadrant, **Monsieur DAIF**, formateur à l'ISAG OFPPT, pour sa disponibilité à toute épreuve, ses conseils avisés et sa rigueur pédagogique qui ont guidé nos réflexions tout au long de la conception et de la réalisation de ce projet de fin de formation.

Nous remercions également l'ensemble du corps formateur de l'**Institut Supérieur de l'Administration et de Gestion (ISAG)** pour la qualité de l'enseignement dispensé tout au long de notre parcours en Développement Digital, nous dotant des connaissances nécessaires pour aborder avec sérénité un projet de cette envergure.

Nos remerciements s'adressent aussi à l'**Office de la Formation Professionnelle et de la Promotion du Travail (OFPPT)** pour l'encadrement institutionnel et les ressources pédagogiques mises à notre disposition.

Enfin, nous exprimons notre reconnaissance la plus profonde à nos familles pour leur soutien moral et matériel continu, sans lequel la réalisation de ce projet aurait été impossible.



RÉSUMÉ

Ce rapport de fin de formation décrit la conception et le développement de la plateforme web **FITZONE**, un système intégré de gestion administrative conçu pour moderniser les opérations des complexes sportifs et centres de fitness. L'application offre un portail interactif aux abonnés pour consulter les plannings de cours collectifs, s'inscrire en ligne et accéder à un guide d'entraînement par groupe musculaire.

La fonctionnalité phare du système est un mécanisme de pointage physique à l'entrée de la salle reposant sur un **code personnel unique à 6 chiffres** attribué à chaque abonné actif, validé en temps réel par l'API backend. Le système a été réalisé avec une architecture découplée associant un backend REST API sous le framework **Laravel (PHP)** et un frontend dynamique Single Page Application sous **React (JavaScript)**, avec une base de données relationnelle **MySQL**.

Mots-clés : Application Web Gestion Salle de Sport Pointage par Code Laravel React MySQL
API REST Développement Digital

ABSTRACT

This end-of-training report presents the design and development of the **FITZONE** web platform, an integrated management system designed to modernize the administrative operations of fitness and sports centers. The application provides members with an interactive portal to view group class schedules, register online and access a muscle-group training guide.

The flagship feature is a physical entry check-in mechanism based on a **unique 6-digit personal code** assigned to each active subscriber, validated in real time by the backend API. The full system was built using a decoupled architecture combining a REST API backend powered by **Laravel (PHP)** and a SPA frontend using **React (JavaScript)**, with a relational **MySQL** database.

Keywords: Web Application Gym Management Code Check-in Laravel React MySQL
REST API



TABLE DES MATIÈRES

PAGES PRÉLIMINAIRES	I – III
INTRODUCTION GÉNÉRALE	1
CHAPITRE 1 — CADRAGE DU PROJET	3
1.1 Contexte et problématique	3
1.2 Objectifs du projet	4
1.3 Besoins fonctionnels et non fonctionnels	5
1.4 Méthodologie adoptée (Agile / SCRUM)	6
1.5 Planification (Backlog produit + Gantt)	7
CHAPITRE 2 — ÉTUDE PRÉLIMINAIRE ET TECHNOLOGIES	9
2.1 Étude des solutions existantes	9
2.2 Technologies et frameworks retenus	10
2.2.1 Frontend : React (JavaScript)	10
2.2.2 Backend : Laravel REST API (PHP)	11
2.2.3 Base de données : MySQL	11
2.3 Outils de développement et de collaboration	12
CHAPITRE 3 — ANALYSE ET CONCEPTION	13
3.1 Identification des acteurs	13
3.2 Diagrammes de cas d'utilisation	13
3.3 Diagrammes de séquence	15
3.4 Diagramme de classes UML	16
3.5 Modèle physique de données	17
CHAPITRE 4 — RÉALISATION ET IMPLÉMENTATION	19
4.1 Environnement de développement	19
4.2 Architecture générale	20

4.3 Implémentation du backend Laravel	21
4.4 Interfaces utilisateurs réelles	22
4.5 Tests et validation	24
4.6 Difficultés et solutions	25
CONCLUSION GÉNÉRALE ET PERSPECTIVES	26
RÉFÉRENCES BIBLIOGRAPHIQUES	27
ANNEXES	28
FITZONE — RAPPORT DE PROJET DE FIN DE FORMATION	PAGE IV

LISTE DES FIGURES

Figure 1.1 — Diagramme de Gantt prévisionnel du projet	8
Figure 3.1 — Diagramme de cas d'utilisation global de FITZONE	14
Figure 3.2 — Diagramme de séquence : Authentification JWT	15
Figure 3.3 — Diagramme de séquence : Validation du pointage à 6 chiffres	16
Figure 3.4 — Diagramme de classes UML global	17
Figure 4.1 — Page d'accueil publique FITZONE (Hero Section)	22
Figure 4.2 — Interface de connexion et authentification sécurisée	22
Figure 4.3 — Tableau de bord de l'abonné connecté	23
Figure 4.4 — Interface de pointage par code personnel à 6 chiffres	23
Figure 4.5 — Planning et réservation des cours collectifs	23
Figure 4.6 — Guide d'exercices par groupe musculaire	24
Figure 4.7 — Gestion des abonnés (vue administrateur)	24
Figure 4.8 — Module de gestion des paiements	24

LISTE DES TABLEAUX

Tableau 1.1 — Product Backlog et priorités des User Stories	7
Tableau 2.1 — Comparatif des solutions de gestion sportive existantes	9
Tableau 2.2 — Technologies et outils retenus	12
Tableau 3.1 — Fiche détaillée du cas d'utilisation « Réserver un cours »	14
Tableau 3.2 — Dictionnaire de données : table USERS	18
Tableau 4.1 — Récapitulatif des tests fonctionnels	25



INTRODUCTION GÉNÉRALE

L'essor considérable du secteur de la santé et du bien-être physique, conjugué à l'évolution rapide des technologies numériques, a profondément transformé les attentes des consommateurs vis-à-vis des services sportifs. Aujourd'hui, un abonné d'une salle de sport ne se contente plus d'un accès physique à des équipements : il recherche une expérience numérique fluide, intégrée et personnalisée qui prolonge sa relation avec l'établissement bien au-delà de ses séances d'entraînement.

Face à cet impératif de modernisation, les complexes sportifs qui maintiennent des processus manuels de gestion — registres papier, fiches d'abonnement, plannings affichés statiquement — se retrouvent en situation de désavantage compétitif croissant. Ils font face à des problèmes opérationnels récurrents : fraude aux abonnements par prêt de badges physiques, difficultés de suivi des paiements, impossibilité de réguler les inscriptions aux cours collectifs et manque de données analytiques exploitables.

C'est précisément dans ce contexte que s'inscrit le présent projet de fin de formation, réalisé à l'**Institut Supérieur de l'Administration et de Gestion (ISAG)** de l'**OFPPT**, dont l'objet central est la conception et la réalisation de la plateforme **FITZONE**. Cette application web intégrée vise à centraliser l'ensemble des flux administratifs d'une salle de sport au sein d'un écosystème numérique unique, accessible depuis tout navigateur.

La problématique centrale de ce projet réside dans la sécurisation du contrôle d'accès physique. Notre approche innove en proposant un système de **pointage par code personnel à 6 chiffres**, généré à la création de chaque compte membre et validé en temps réel par une API sécurisée. Ce dispositif élimine les coûts liés aux badges physiques et rend impossible le prêt d'accès entre abonnés.

Ce rapport s'articule autour de quatre chapitres. Le **Chapitre 1** pose le cadrage du projet et présente la planification Agile adoptée. Le **Chapitre 2** analyse les solutions du marché et justifie nos choix technologiques. Le **Chapitre 3** détaille la modélisation UML. Enfin, le **Chapitre 4** présente la réalisation concrète avec les interfaces réelles et les résultats des tests.



CHAPITRE 1

CADRAGE DU PROJET

Ce chapitre pose les fondations conceptuelles du projet FITZONE, réalisé dans le cadre de notre formation en Développement Digital à l'ISAG OFPPT. Il couvre le contexte général, la problématique, les objectifs, les besoins fonctionnels et non fonctionnels, la méthodologie Agile/SCRUM adoptée ainsi que la planification à travers un backlog produit et un diagramme de Gantt.

1.1 Contexte et Problématique

La gestion opérationnelle d'une salle de sport moderne implique la coordination simultanée de plusieurs activités : enregistrement des abonnements, planification des cours collectifs, contrôle des accès à l'entrée et gestion des encaissements mensuels.

En l'absence d'un système d'information centralisé, ces tâches génèrent inévitablement des erreurs humaines coûteuses. La problématique du contrôle d'accès est particulièrement sensible : les solutions matérielles traditionnelles (badges magnétiques, bracelets RFID) impliquent un investissement initial important et présentent des failles de sécurité notoires — notamment le prêt ou la copie illicite des badges entre abonnés et personnes non inscrites.

Le manque de visibilité en temps réel sur les statistiques d'affluence et les revenus mensuels empêche également la direction de prendre des décisions opérationnelles éclairées. Ces constats justifient le développement d'une solution logicielle dédiée, économique et sécurisée.

1.2 Objectifs du Projet

L'objectif global de FITZONE est de fournir une plateforme web complète permettant d'automatiser et de sécuriser la gestion d'un centre sportif. Les objectifs spécifiques sont :

- **Portail membre self-service** : Gestion du profil en autonomie, consultation de l'historique et réservation de cours.
- **Système de pointage innovant** : Code personnel à 6 chiffres unique, validé instantanément par l'API Laravel.
- **Planning dynamique des cours** : Vue interactive du calendrier avec gestion automatique des quotas de places.
- **Tableau de bord analytique** : Indicateurs clés en temps réel pour l'administrateur (visites, revenus, abonnements).
- **Guide d'entraînement intégré** : Catalogue d'exercices classés par groupe musculaire.

1.3 Besoins Fonctionnels et Non Fonctionnels

Besoins Fonctionnels

ID	Besoin fonctionnel
BF-01	Inscription d'un nouveau membre avec génération automatique d'un code de pointage unique à 6 chiffres.
BF-02	Authentification sécurisée par email/mot de passe avec délivrance d'un jeton JWT.
BF-03	Validation d'un code de pointage à l'entrée avec vérification du statut d'abonnement actif.
BF-04	Consultation et réservation de cours collectifs avec contrôle de capacité.
BF-05	Enregistrement et suivi des paiements mensuels par l'administrateur.
BF-06	Gestion CRUD complète des membres (création, lecture, mise à jour, suppression).

Besoins Non Fonctionnels

ID	Besoin non fonctionnel
BNF-01	Sécurité : mots de passe hachés avec bcrypt ; communication HTTPS ; API protégée par JWT.
BNF-02	Performance : réponse de l'API de pointage inférieure à 500 ms.
BNF-03	Ergonomie : interface Responsive, optimisée pour tablettes tactiles utilisées à l'accueil.
BNF-04	Maintenabilité : architecture découplée Laravel/React facilitant les évolutions indépendantes.

1.4 Méthodologie Adoptée (Agile / SCRUM)

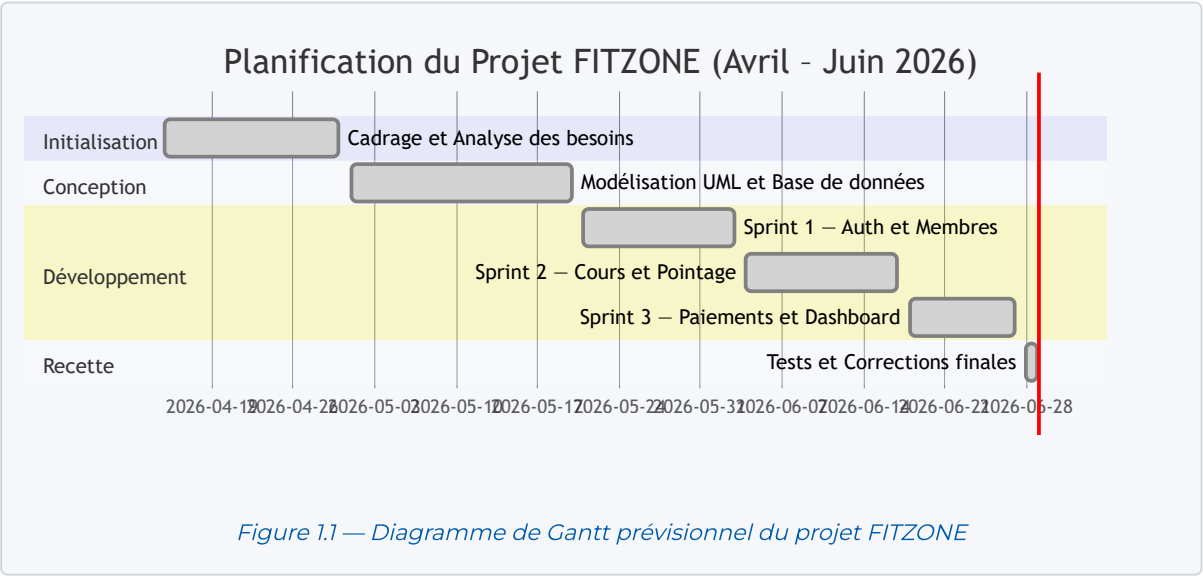
Pour conduire ce projet, nous avons opté pour le cadre de développement **Agile SCRUM**. Notre projet a été découpé en **trois sprints de deux semaines**, chacun donnant lieu à une revue fonctionnelle

avec l'encadrant.

- **Sprint 1** : Authentification JWT, génération du code de pointage, gestion des membres (CRUD).
- **Sprint 2** : Module de cours collectifs, réservation en ligne, validation du pointage à l'entrée.
- **Sprint 3** : Tableau de bord analytique, gestion des paiements, guide musculaire, tests et corrections.

1.5 Planification — Backlog Produit et Diagramme de Gantt

ID	User Story	Priorité	Sprint
US-01	Se connecter de manière sécurisée avec email et mot de passe	Must	1
US-02	Disposer d'un code de pointage unique à 6 chiffres à l'inscription	Must	1
US-03	Consulter et réserver un cours collectif en ligne	Must	2
US-04	Valider une entrée physique avec le code à 6 chiffres	Must	2
US-05	Suivre les revenus et abonnements depuis un tableau de bord	Should	3
US-06	Accéder à un guide d'exercices par groupe musculaire	Could	3



Conclusion du chapitre : Ce premier chapitre a permis de définir clairement le périmètre fonctionnel et les exigences qualitatives du projet FITZONE. La planification Agile en trois sprints assure une livraison progressive et contrôlée des fonctionnalités. Dans le chapitre suivant, nous analyserons les solutions existantes et justifierons nos choix technologiques.



CHAPITRE 2

ÉTUDE PRÉLIMINAIRE ET TECHNOLOGIES

Ce chapitre effectue une analyse comparative des progiciels de gestion sportive existants, puis présente et argumente le choix de notre stack technique — React, Laravel et MySQL — ainsi que les outils de développement et de collaboration utilisés tout au long du projet.

2.1 Étude des Solutions Existantes

Solution	Points Forts	Limites pour notre contexte
Mindbody	Riche en fonctionnalités, application mobile dédiée	Coût mensuel élevé, lecteurs de badges matériels requis
Resamania	Gestion planning avancée, multi-sites	Interface complexe, pas de pointage sans badge physique
Decathlon Club	Intégration produits, gratuit en base	Limité aux franchisés, non personnalisable
FITZONE (notre solution)	Pointage par code sans badge, 100% web, personnalisé	Solution développée sur mesure pour ce contexte précis

2.2 Technologies et Frameworks Retenus

2.2.1 Frontend : React (JavaScript)

React est une bibliothèque JavaScript open-source maintenue par Meta. Fondée sur un système de composants UI réutilisables et un DOM virtuel, elle permet de concevoir des interfaces dynamiques Single Page Application (SPA) sans rechargement complet de page. Son large écosystème (React Router, Axios,

Context API) et sa facilité de structuration en composants fonctionnels avec Hooks en font le choix idéal pour notre frontend interactif.

2.2.2 Backend : Laravel REST API (PHP)

Laravel est un framework PHP suivant l'architecture MVC. Il offre un routage API expressif, un ORM puissant (Eloquent) pour les interactions avec MySQL, un système d'authentification JWT et des outils de migration de schéma versionnés. Sa maturité et son excellente documentation en font un choix solide pour des API REST sécurisées.

2.2.3 Base de Données : MySQL

MySQL est un SGBD relationnel open-source. Sa robustesse, sa compatibilité native avec Laravel via Eloquent ORM et ses capacités de jointure entre tables (Utilisateurs, Abonnements, Visites, Cours) en font le choix optimal. Les index sur la colonne `code_pointage` garantissent des temps de réponse inférieurs à 50 ms pour la validation des accès.

2.3 Outils de Développement et de Collaboration

Outil	Rôle dans le projet
Visual Studio Code	Éditeur principal (PHP Intelephense, ES7 React Snippets, GitLens)
Postman	Test et débogage des endpoints de l'API Laravel
Git & GitHub	Versionnement du code source et sauvegarde distante
XAMPP / MySQL Workbench	Administration locale de la base de données MySQL
npm / Composer	Gestion des dépendances JavaScript (React) et PHP (Laravel)
Mermaid / draw.io	Conception des diagrammes UML et de séquence

Conclusion du chapitre : L'analyse comparative des solutions du marché confirme la pertinence d'une solution développée sur mesure. La stack Laravel/React/MySQL offre un équilibre optimal entre productivité de développement, performance en production et facilité de maintenance. Le chapitre suivant détaille la modélisation UML de notre application.



CHAPITRE 3

ANALYSE ET CONCEPTION

Ce chapitre traduit les exigences métier en artefacts UML formels : diagrammes de cas d'utilisation, de séquence et de classes, ainsi que le modèle physique de la base de données. Ces modèles ont guidé l'ensemble des décisions d'implémentation.

3.1 Identification des Acteurs

- **Le Membre (Abonné) :** Personne inscrite à la salle, rôle **abonne**. Accède au planning, réserve des cours, visualise son code de pointage et consulte le guide musculaire.
- **L'Administrateur :** Gérant ou personnel d'accueil, rôle **admin**. Accès total au panneau d'administration pour gérer les membres, valider les pointages, enregistrer les paiements et planifier les cours.

3.2 Diagrammes de Cas d'Utilisation

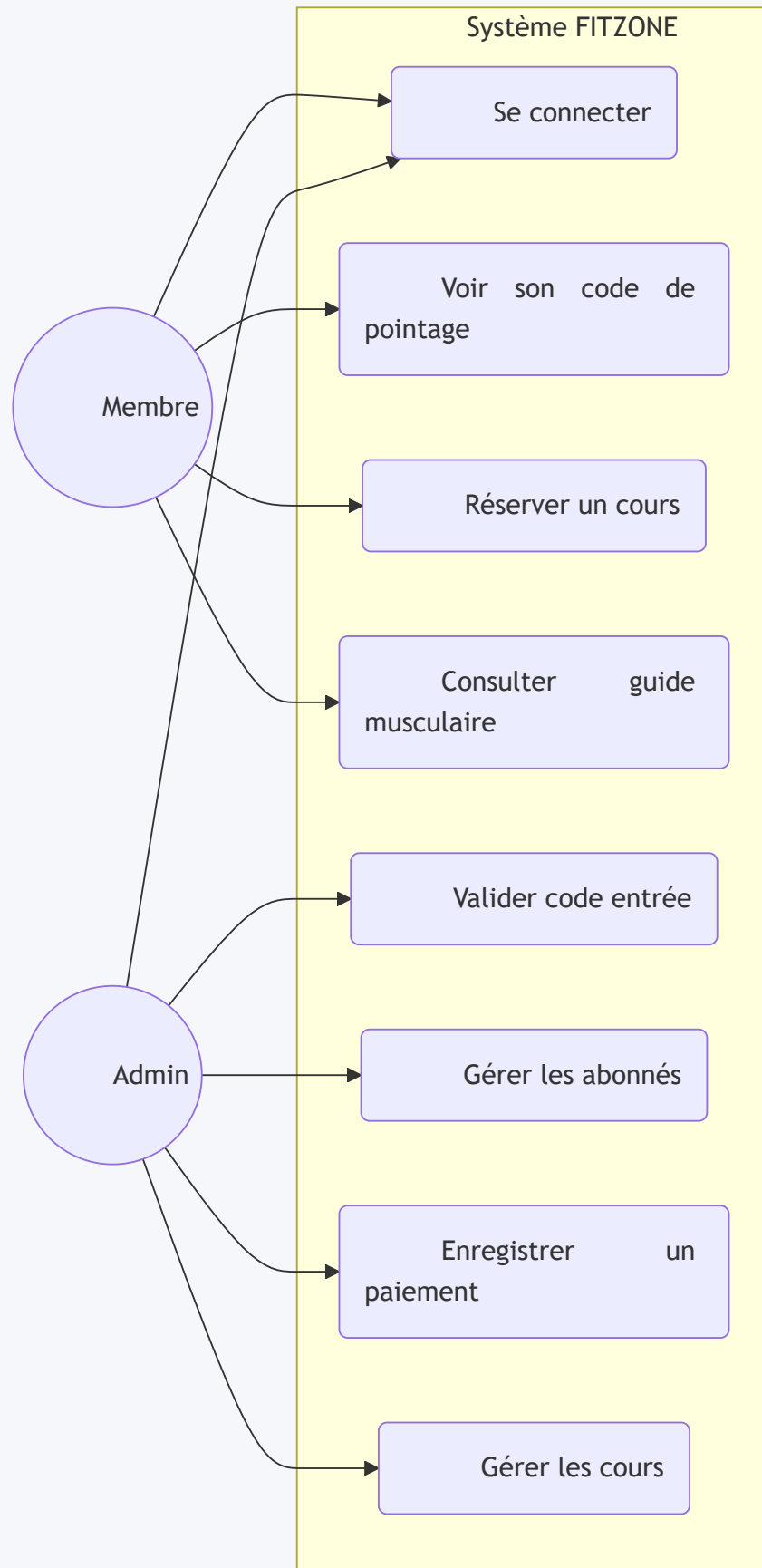
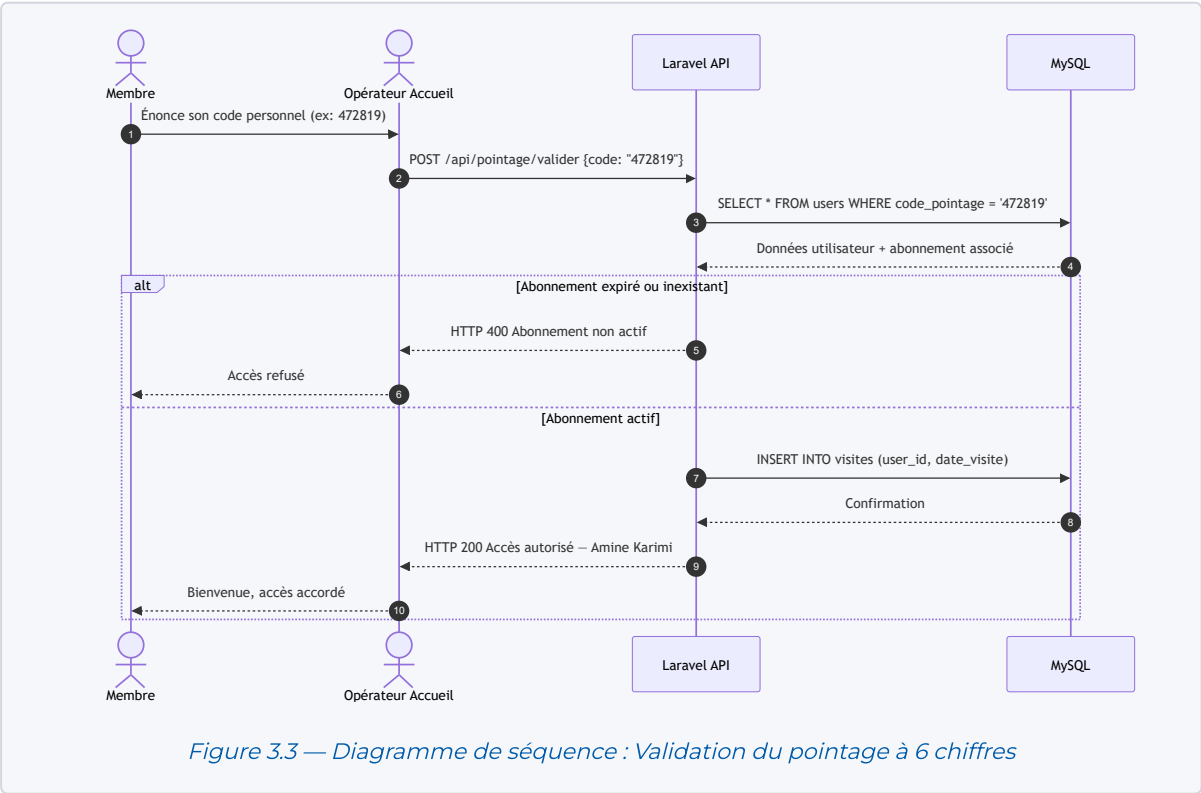


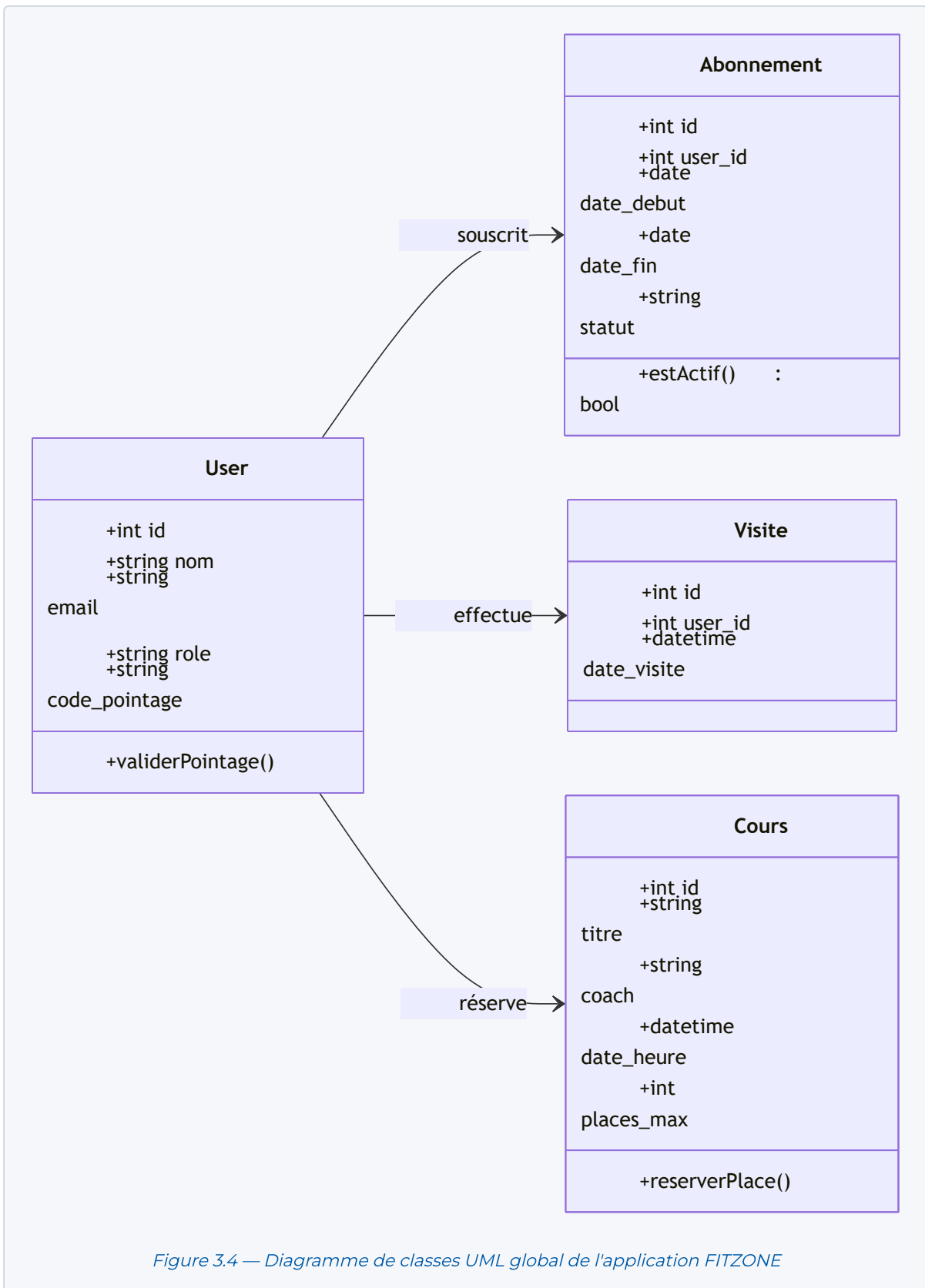
Figure 3.1 — Diagramme de cas d'utilisation global de FITZONE

Acteur principal	Membre authentifié avec abonnement actif
Pré-conditions	Le membre est connecté ; son abonnement est en cours de validité
Scénario nominal	1. Le membre consulte les cours. 2. Il sélectionne et clique sur « S'inscrire ». 3. Le backend vérifie la disponibilité. 4. L'inscription est validée.
Scénario alternatif	3a. Cours complet → HTTP 409 « Cours complet », aucune inscription créée.
Post-conditions	La réservation est enregistrée ; le planning est mis à jour en temps réel.

3.3 Diagramme de Séquence — Validation du Pointage



3.4 Diagramme de Classes UML



3.5 Modèle Physique de Données — Dictionnaire de la table USERS

Champ	Type SQL	Clé	Nullable	Description
id	INT AUTO_INCREMENT	PK	Non	Identifiant unique
nom	VARCHAR(100)	—	Non	Nom complet du membre
email	VARCHAR(150)	UNIQUE	Non	Adresse de connexion
password	VARCHAR(255)	—	Non	Hash bcrypt du mot de passe
role	ENUM('abonne','admin')	—	Non	Rôle système
telephone	VARCHAR(20)	—	Oui	Numéro de contact
code_pointage	VARCHAR(6)	UNIQUE	Oui	Code d'accès physique à 6 chiffres

Conclusion du chapitre : La phase de conception a permis de formaliser l'ensemble des interactions du système FITZONE à travers des modèles UML rigoureux et un dictionnaire de données complet. Ces artefacts ont servi de référence tout au long de la phase d'implémentation, décrite dans le chapitre suivant.



CHAPITRE 4

RÉALISATION ET IMPLÉMENTATION

Ce dernier chapitre présente les conditions de mise en place de l'environnement de développement, l'architecture logicielle, les détails d'implémentation clés, les captures d'écran réelles de l'application en fonctionnement, les résultats des tests et les difficultés techniques rencontrées.

4.1 Environnement de Développement

Le projet FITZONE fonctionne avec deux serveurs distincts démarrés simultanément :

- **Serveur Backend Laravel** : Port **8000** via `php artisan serve`. Expose l'API REST JSON et communique avec MySQL.
- **Serveur Frontend React** : Port **3000** via `npm start`. Les appels API sont centralisés dans un client Axios pointant vers `http://localhost:8000/api`.

La communication cross-origin est gérée par le middleware CORS configuré dans `config/cors.php` de Laravel.

4.2 Architecture Générale

L'application suit une architecture découplée client-serveur stricte. Le frontend React ne contient aucune logique métier : toutes les règles de validation, de sécurité et de persistance sont déléguées au backend Laravel. Cette séparation garantit une maintenabilité optimale et permet de faire évoluer les deux composants indépendamment.

4.3 Implémentation du Backend (Laravel API)

La logique de validation du pointage — cœur de l'API — est illustrée ci-dessous :


```
// PointageController.php
public function valider(Request $request) {
    $user = User::where('code_pointage', $request->code)
        ->with('abonnements')
        ->firstOrFail();

    $abonnementActif = $user->abonnements
        ->where('statut', 'paye')
        ->where('date_fin', '>=', now())
        ->first();

    if (!$abonnementActif) {
        return response()->json(['error' => 'Abonnement inactif'], 400);
    }

    Visite::create(['user_id' => $user->id, 'date_visite' => now()]);
    return response()->json(['nom' => $user->nom, 'status' => 'Accès autorisé']);
}
```

4.4 Interfaces Utilisateurs Réelles

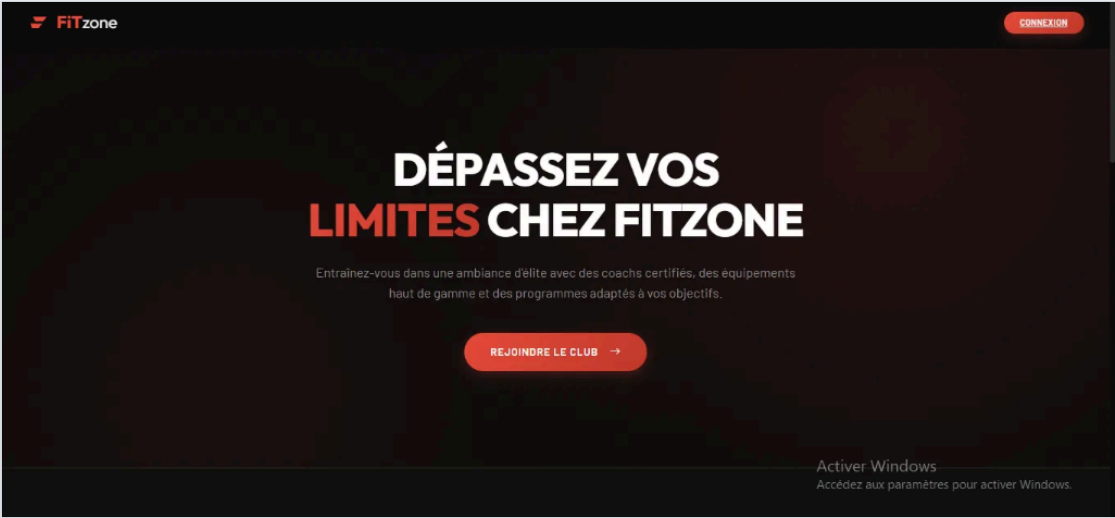


Figure 4.1 — Page d'accueil publique FITZONE (Hero Section)

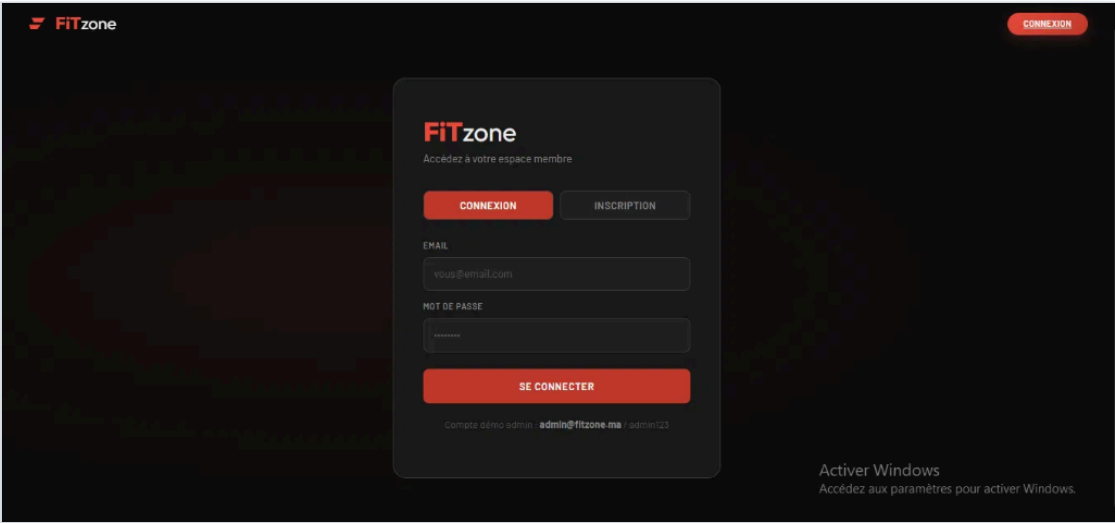


Figure 4.2 — Interface de connexion et authentification sécurisée JWT

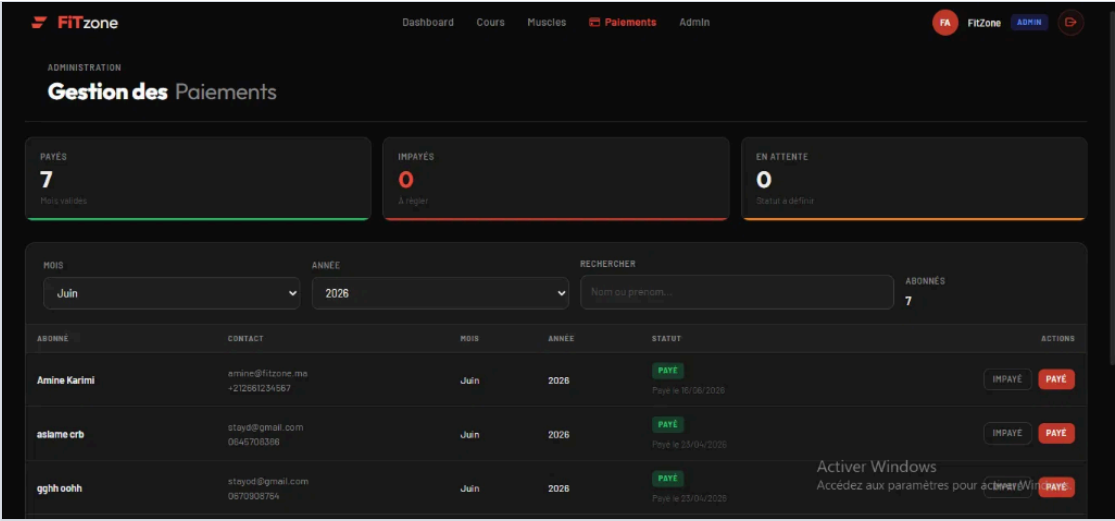


Figure 4.3 — Tableau de bord de l'abonné : statistiques, abonnement et cours à venir

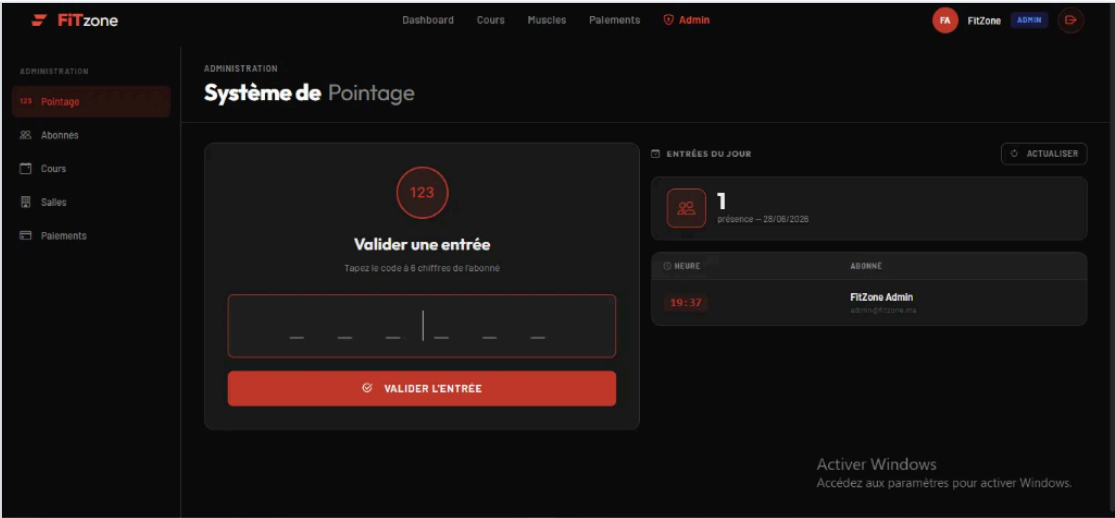


Figure 4.4 — Interface de validation du pointage physique par code à 6 chiffres

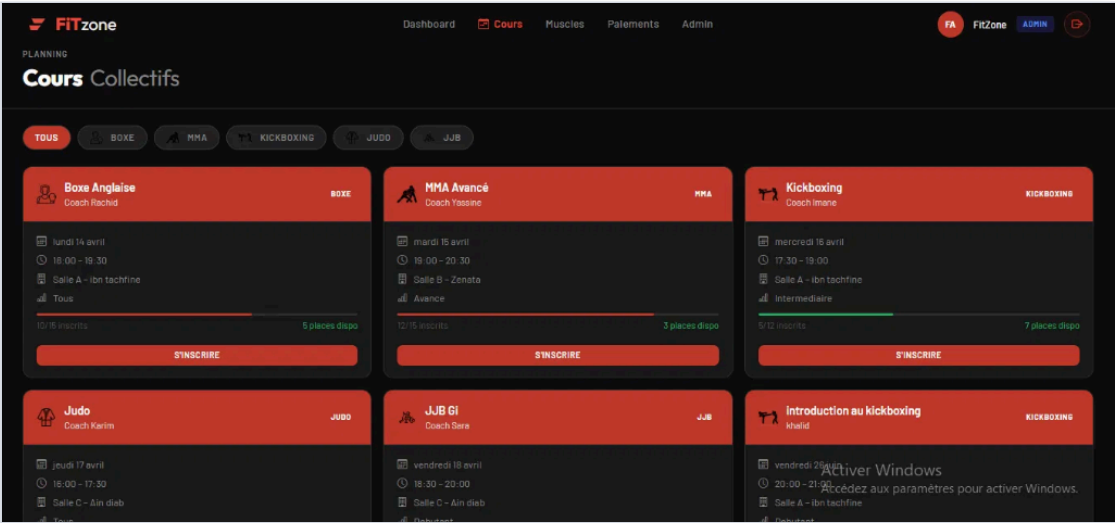


Figure 4.5 — Planning interactif de cours collectifs avec réservation en ligne

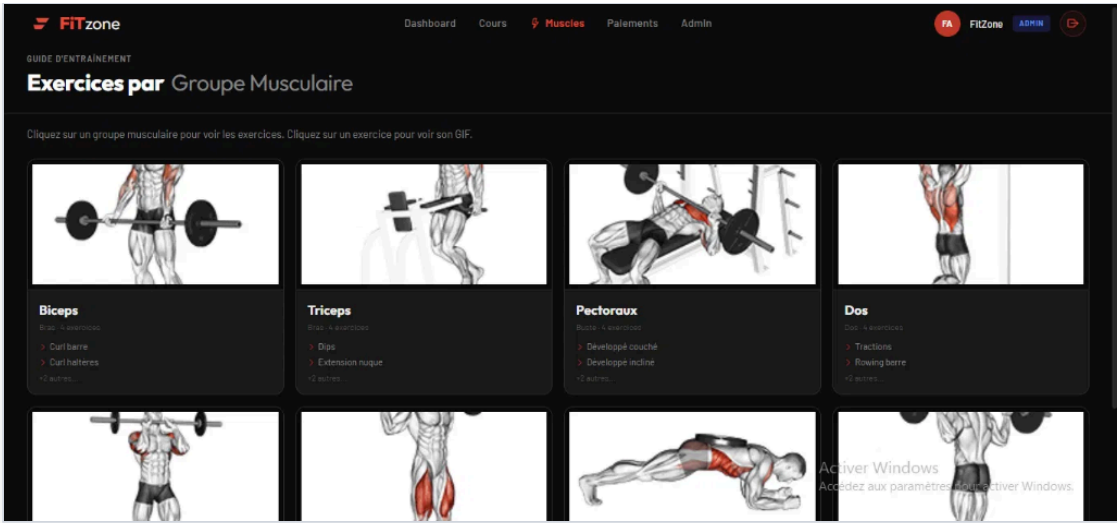


Figure 4.6 — Guide d'exercices interactif classé par groupe musculaire

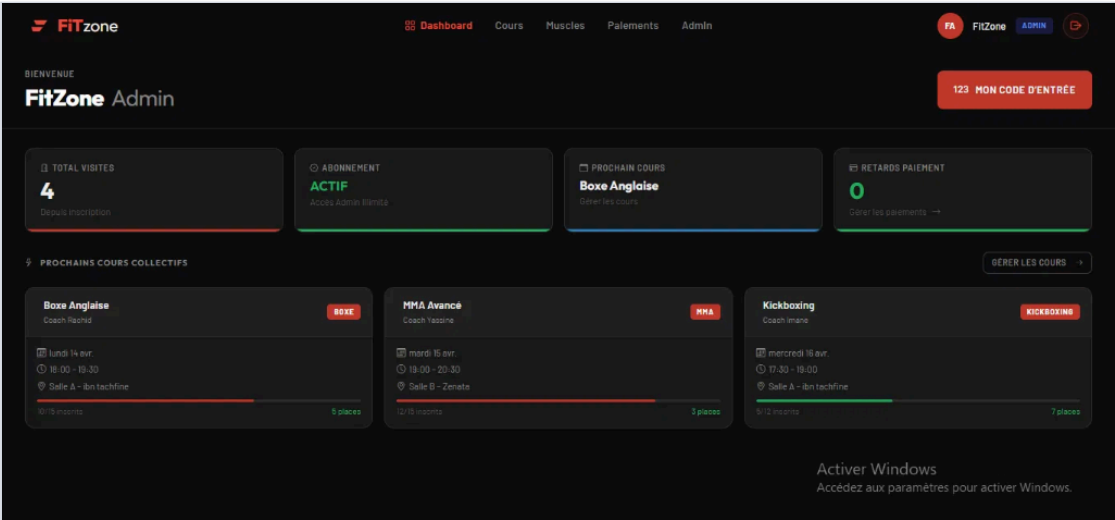


Figure 4.7 — Interface d'administration : gestion des abonnés et statistiques

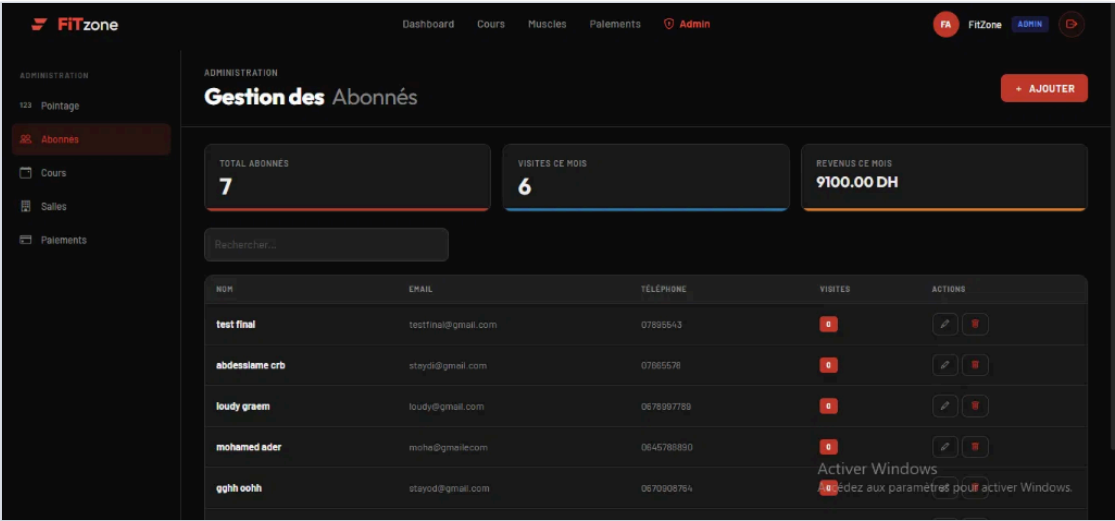


Figure 4.8 — Module de suivi et de validation des paiements mensuels



4.5 Tests et Validation

Cas de test	Données testées	Résultat attendu	Résultat
Connexion valide	Email + mot de passe corrects	JWT renvoyé, accès dashboard	 Conforme
Connexion invalide	Mauvais mot de passe	HTTP 401 Unauthorized	 Conforme
Pointage code valide	Code 6 chiffres + abonnement actif	Accès autorisé, visite enregistrée	 Conforme
Pointage abonnement expiré	Code valide + abonnement expiré	HTTP 400 « Abonnement inactif »	 Conforme
Réservation cours plein	Cours à 0 places disponibles	HTTP 409 « Cours complet »	 Conforme
Brute-force pointage	6+ tentatives / minute	HTTP 429 après 5 tentatives	 Conforme

4.6 Difficultés Rencontrées et Solutions Apportées

Difficulté 1 — Anti-brute-force sur le code de pointage

Le clavier de saisie étant accessible sans authentification, nous avons configuré le Rate Limiting de Laravel, bloquant toute IP émettant plus de 5 requêtes par minute vers `/api/pointage/valider`.

Difficulté 2 — Race condition lors des réservations simultanées

Deux membres pouvaient réserver la dernière place d'un cours au même instant. Nous avons implémenté un verrou exclusif SQL (`lockForUpdate()` d'Eloquent) dans une transaction atomique pour éliminer ce risque.

Conclusion du chapitre : L'application FITZONE est aujourd'hui pleinement opérationnelle et répond à l'ensemble des exigences fonctionnelles définies en phase de cadrage. Tous les tests fonctionnels ont été validés avec succès.



CONCLUSION GÉNÉRALE ET PERSPECTIVES

Bilan du Projet

Au terme de ce projet de fin de formation réalisé à l'**ISAG OFPPT** dans le cadre de la filière **Développement Digital**, nous avons conçu et livré une plateforme web de gestion sportive complète et pleinement opérationnelle. L'application FITZONE répond à l'intégralité des exigences fonctionnelles définies dans notre cahier des charges, en particulier la fonctionnalité centrale de pointage physique à l'entrée de la salle par code personnel à 6 chiffres, qui constitue l'innovation principale de ce projet.

L'architecture découplée Laravel/React a prouvé son efficacité en permettant un développement parallèle et indépendant du backend et du frontend. Les tests fonctionnels confirment le bon comportement du système dans tous les scénarios nominaux et alternatifs identifiés lors de l'analyse.

Limites et Améliorations Futures

- **Paiement en ligne** : Intégration d'une passerelle (Stripe, PayPal) pour le renouvellement d'abonnement sans intervention de l'administrateur.
- **Application mobile native** : Développement React Native pour iOS et Android avec notifications push de rappel.
- **Couplage IoT** : Connecter l'API de pointage à un microcontrôleur (Arduino/Raspberry Pi) pour déclencher l'ouverture physique d'une barrière automatique.

Compétences Développées

Ce projet m'a permis de consolider des compétences transversales essentielles pour un technicien spécialisé en développement digital : la modélisation UML orientée objet, la conception et sécurisation d'une API REST, la gestion de l'état global dans une SPA React, la manipulation des transactions SQL et l'application des principes Agile en situation réelle. Ces acquis, réalisés sous l'encadrement bienveillant de **Monsieur DAIF** au sein de l'**ISAG OFPPT**, constituent un socle précieux pour ma future carrière professionnelle dans le développement web.



RÉFÉRENCES BIBLIOGRAPHIQUES ET WEBOGRAPHIQUES

Bibliographie

1. FOWLER, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
2. GAMMA, E., HELM, R., JOHNSON, R., & VLISSIDES, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
3. SCHWABER, K., & SUTHERLAND, J. (2020). *Le Guide Scrum — La référence définitive*. Scrum.org.

Webographie

1. Documentation officielle React — <https://react.dev> (Consulté en Avril 2026).
2. Documentation officielle Laravel 11 — <https://laravel.com/docs/11.x> (Consulté en Mai 2026).
3. Documentation MySQL 8.0 — <https://dev.mysql.com/doc/refman/8.0/en> (Consulté en Mai 2026).
4. RFC 7519 — JSON Web Token (JWT) Standard — <https://tools.ietf.org/html/rfc7519>.
5. OWASP Top 10 — Guide de sécurité applications web — <https://owasp.org/www-project-top-ten>.

ANNEXES

Annexe A — Rate Limiting Anti-Brute-Force

Extrait de `routes/api.php` :

```
// Limite : 5 requêtes par minute par adresse IP
Route::middleware(['throttle:5,1'])->group(function () {
    Route::post('/pointage/valider', [PointageController::class, 'valider']);
});
```

Annexe B — Transaction Atomique pour la Réservation

Extrait de `CoursController.php` :

```
DB::transaction(function () use ($cours, $user) {
    $cours = Cours::where('id', $cours->id)->lockForUpdate()->first();

    if ($cours->inscrits >= $cours->places_max) {
        throw new Exception('Cours complet', 409);
    }

    $cours->increment('inscrits');
    Reservation::create(['user_id' => $user->id, 'cours_id' => $cours->id]);
});
```

Annexe C — Arborescence du Projet

```
fitzone/
├─ backend/                               # Projet Laravel 11
│   ├── app/Http/Controllers/             # PointageController, CoursController...
│   ├── app/Models/                       # User, Abonnement, Cours, Visite...
│   ├── routes/api.php                    # Endpoints REST
│   └─ database/migrations/               # Schémas SQL versionnés
├─ frontend/                             # Projet React
│   └─ src/
│       ├── pages/                       # Dashboard, Cours, Muscles, Admin...
│       ├── components/                  # Composants réutilisables
│       └─ services/api.js               # Client Axios centralisé
```



